WEB & APP DEVELOPMENT

Flutter

Build beautiful, natively compiled apps for mobile, web, and desktop from a single Dart codebase.

flutter dart widgets state-management cross-platform mobile ui

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. What Flutter Is & Why It Matters
2. Setup & Getting Started
3. Dart Essentials
4. Everything Is a Widget
5. Layout: Rows, Columns, and Constraints
6. Material Widgets & Theming
7. Navigation & Routing
8. Async, Futures & Streams
9. Calling REST APIs
10. State Management Overview
11. Worked Mini-Example: A Posts Screen
12. Cheat Sheet & Comparison
13. Best Practices & Common Pitfalls
14. Interview Questions & Answers
15. Further Learning

What Flutter Is & Why It Matters

Flutter is Google's open-source UI toolkit for building natively compiled applications for mobile (iOS/Android), web, desktop (Windows/macOS/Linux), and embedded devices — all from a single `Dart` codebase.

Unlike older cross-platform frameworks that wrap native widgets or run inside a webview, Flutter ships its own high-performance rendering engine (Impeller, which replaced Skia as the default). Flutter *draws every pixel itself* onto a canvas. The consequence is pixel-identical UI across platforms, 60/120fps animations, and no dependency on the OS widget library.

SINGLE CODEBASE

One Dart project targets 6 platforms. Share business logic and UI.

HOT RELOAD

Inject code changes into the running app in under a second while preserving state.

EVERYTHING IS A WIDGET

Layout, styling, padding, gestures, animation — all composed from immutable widget objects.

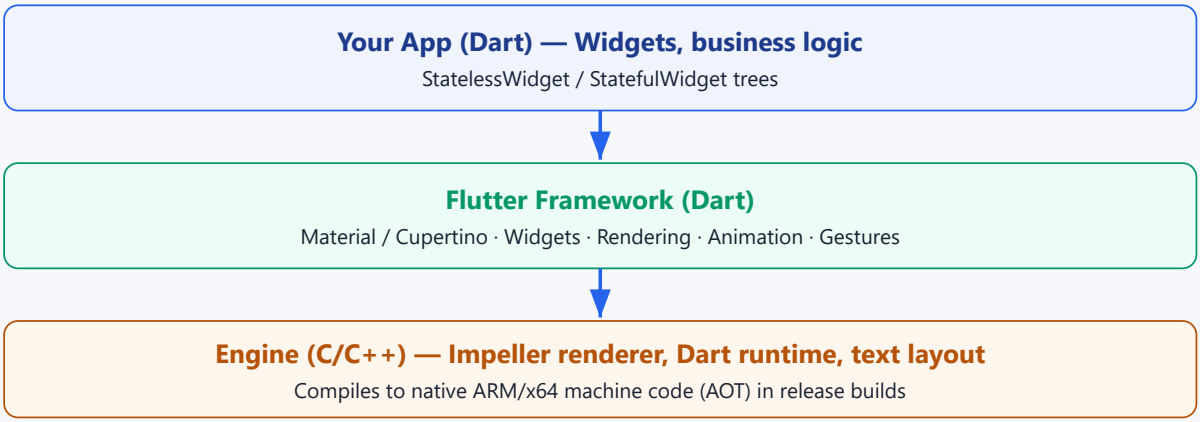


Fig 1. The Flutter architecture: your widgets sit on the framework, which sits on the native engine.

Mental model

A Flutter UI is a **function of state**: `UI = f(state)`. You never imperatively mutate widgets ("set this label's text"). Instead you change state and let the framework rebuild the affected part of the tree.

Setup & Getting Started

Install the Flutter SDK, then verify your toolchain with `flutter doctor`, which checks for Android Studio, Xcode, and connected devices.

BASH

```
# Verify the install and all platform toolchains
flutter doctor -v

# Create a new app (package name uses reverse-DNS)
flutter create --org com.example my_app
cd my_app

# List connected devices / emulators
flutter devices

# Run on a chosen device (hot reload enabled by default in debug)
flutter run -d chrome          # web
flutter run -d emulator-5554   # Android emulator

# Add a dependency from pub.dev
flutter pub add http
flutter pub get

# Build release artifacts
flutter build apk --release
flutter build ipa                # iOS (needs macOS + Xcode)
flutter build web --release
```

✦ **Hot reload vs hot restart**

Press `r` in the terminal for hot reload (keeps state), `R` for hot restart (resets state), and `q` to quit. Hot reload cannot pick up changes to `main()`, global initializers, or `enum`/type changes — use hot restart there.

Project layout

Path	Purpose
`lib/main.dart`	Entry point; contains `main()` and the root widget.
`pubspec.yaml`	Dependencies, assets, fonts, SDK constraints.
`test/`	Unit & widget tests.
`android/ ios/ web/ ...`	Per-platform host projects (native config).

Dart Essentials

Dart is a type-safe, null-safe, object-oriented language with a C-style syntax. You cannot write Flutter without it, so master these fundamentals first.

DART

```dart
// Variables: `var` infers, explicit types are optional but recommended for APIs.
var name = 'Ada';            // String (inferred)
final year = 1815;           // runtime constant, set once
const pi = 3.14159;          // compile-time constant
int? maybeNull;              // nullable — null safety is on by default

// String interpolation
print('Hello $name, born in $year. Next year: ${year + 1}');

// Collections with literals + spread + collection-if
final nums = [1, 2, 3];
final more = [0, ...nums, if (nums.length > 2) 99];  // [0,1,2,3,99]
final set = {'a', 'b'};
final map = {'one': 1, 'two': 2};

// Functions: arrow syntax, named + optional params with defaults
int add(int a, int b) => a + b;
String greet(String who, {String greeting = 'Hi'}) => '$greeting, $who';
greet('Sam');                        // "Hi, Sam"
greet('Sam', greeting: 'Hello');     // named argument

// Null-aware operators
String display(String? s) => s ?? 'unknown';   // ?? default
int? len = maybeNull?.length;                   // ?. safe navigation
```

Classes, immutability, and records

DART

```dart
class User {
  final String name;
  final int age;
  // Named constructor params with `required`
  const User({required this.name, required this.age});

  // Named constructor
  User.guest() : name = 'Guest', age = 0;

  bool get isAdult => age >= 18;              // getter
  User copyWith({String? name, int? age}) =>  // common immutable pattern
      User(name: name ?? this.name, age: age ?? this.age);
}

// Records (Dart 3+): lightweight anonymous tuples
(String, int) parse() => ('ok', 200);
final (label, code) = parse();   // destructuring

// Pattern matching / switch expressions (Dart 3+)
String classify(int n) => switch (n) {
  0 => 'zero',
  < 0 => 'negative',
  _ => 'positive',
};
```

> **Why immutability matters**
>
> Widgets are immutable. The `copyWith` pattern above is everywhere in Flutter — it is how you produce a modified copy of state objects without mutation, which keeps rebuilds predictable.

Everything Is a Widget

Widgets are immutable descriptions of part of the UI. Flutter builds three parallel trees to turn them into pixels.

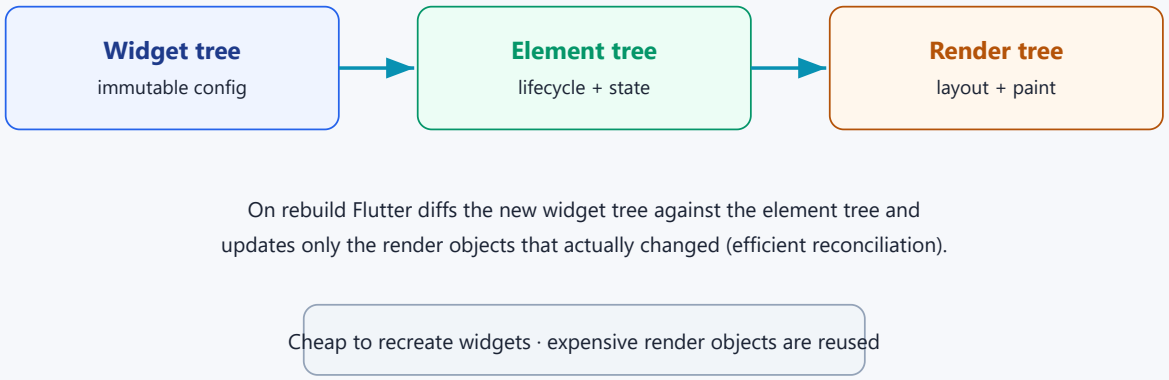


Fig 2. Three trees. Widgets are throwaway blueprints; Elements and RenderObjects persist across rebuilds.

StatelessWidget

Use when the UI depends only on its configuration and never changes on its own.

DART

```dart
import 'package:flutter/material.dart';

class Greeting extends StatelessWidget {
  final String name;
  const Greeting({super.key, required this.name});

  @override
  Widget build(BuildContext context) {
    return Text('Hello, $name!', style: Theme.of(context).textTheme.headlineSmall);
  }
}
```

StatefulWidget

Use when the widget holds mutable state that changes over its lifetime (counters, form input, toggles, animations).

DART

```dart
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() => _CounterState();
}

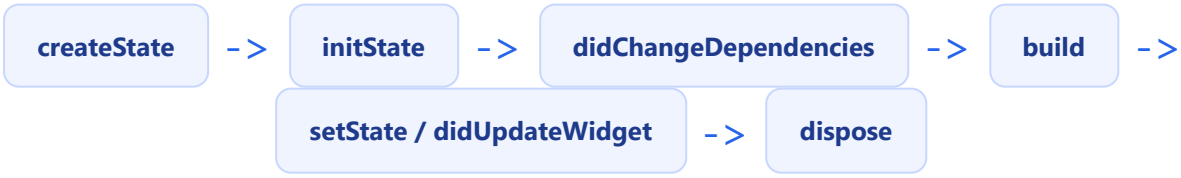
class _CounterState extends State<Counter> {
  int _count = 0;

  void _increment() {
    // setState tells the framework this State is dirty and must rebuild.
    setState(() => _count++);
  }

  @override
  Widget build(BuildContext context) {
    return Column(
      mainAxisSize: MainAxisSize.min,
      children: [
        Text('Count: $_count', style: const TextStyle(fontSize: 24)),
        ElevatedButton(onPressed: _increment, child: const Text('Add')),
      ],
    );
  }
}
```

> ⚠ **setState gotcha**
>
> Never call `setState` inside `build()`, inside a loop, or after the widget is disposed. It only schedules a rebuild of *this* State — it does not update parents or siblings.

State lifecycle



Method	When	Typical use
`initState()`	Once, before first build	Subscribe to streams, init controllers.
`build()`	Every rebuild	Return the widget subtree. Keep it pure & fast.
`didUpdateWidget()`	Parent rebuilds with new config	React to changed widget fields.
`dispose()`	Once, on removal	Cancel timers/subscriptions, dispose controllers.

✕ **Memory leaks**

Any `AnimationController`, `TextEditingController`, `ScrollController`, `StreamSubscription`, or `Timer` you create in `initState` MUST be released in `dispose`. Forgetting this is the #1 source of leaks and "setState called after dispose" errors.

Layout: Rows, Columns, and Constraints

Flutter layout follows one sentence: **constraints go down, sizes go up, parent sets position**. A parent passes constraints (min/max width & height) to each child; the child chooses its size within them; the parent positions it.

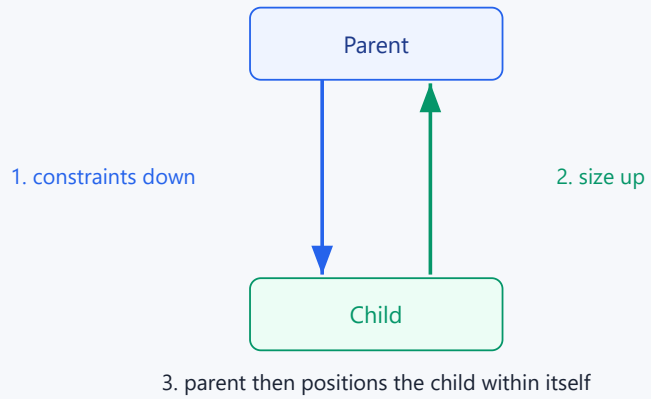


Fig 3. The constraint-propagation layout algorithm — the key to debugging overflow errors.

DART

```dart
Column(
  crossAxisAlignment: CrossAxisAlignment.start, // horizontal in a Column
  mainAxisAlignment: MainAxisAlignment.spaceBetween, // vertical
  children: [
    const Text('Title'),
    Row(children: [
      const Icon(Icons.star),
      const SizedBox(width: 8),            // fixed-size spacer
      Expanded(                            // takes remaining horizontal space
        child: Text('Subtitle', overflow: TextOverflow.ellipsis),
      ),
    ]),
    Padding(
      padding: const EdgeInsets.all(16),
      child: Container(
        decoration: BoxDecoration(
          color: Colors.blue.shade50,
          borderRadius: BorderRadius.circular(12),
        ),
        child: const Text('Boxed'),
      ),
    ),
  ],
)
```

EXPANDED / FLEXIBLE

Divide leftover space along the main axis of a Row/Column. Use `flex:` to weight.

STACK / POSITIONED

Overlap children; `Positioned` pins to edges for badges, FABs, overlays.

SIZEDBOX / PADDING

Spacing and fixed dimensions without a heavyweight `Container`.

LAYOUTBUILDER

Build differently based on the incoming constraints — the basis of responsive design.

⚠ **"RenderFlex overflowed"**

This yellow-and-black striped error means a child wanted more space than the constraint allowed (e.g. a long `Text` in a `Row`). Fix it by wrapping the child in `Expanded` / `Flexible`, or making the content scrollable.

Material Widgets & Theming

Flutter ships `Material 3` (Google) and `Cupertino` (iOS) design systems. `MaterialApp` provides navigation, theming, and localization scaffolding.

DART

```dart
void main() => runApp(const MyApp());

class MyApp extends StatelessWidget {
  const MyApp({super.key});
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Demo',
      theme: ThemeData(
        useMaterial3: true,
        colorScheme: ColorScheme.fromSeed(seedColor: Colors.indigo),
      ),
      home: const HomePage(),
    );
  }
}

class HomePage extends StatelessWidget {
  const HomePage({super.key});
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Home')),
      body: const Center(child: Text('Body content')),
      floatingActionButton: FloatingActionButton(
        onPressed: () {},
        child: const Icon(Icons.add),
      ),
    );
  }
}
```

Widget	Role
`Scaffold`	Page skeleton: app bar, body, FAB, drawer, bottom nav.
`AppBar`	Top bar with title, actions, leading icon.
`ListView / ListView.builder`	Scrolling list; `.builder` lazily builds only visible items.
`Card / ListTile`	Elevated surface and a standard row (leading/title/subtitle/trailing).
`TextField / Form`	Input with validation via `TextFormField` + `GlobalKey<FormState>`.
`showDialog / SnackBar / BottomSheet`	Transient overlays and feedback.

Navigation & Routing

Flutter models navigation as a stack of routes. The imperative `Navigator` API is fine for simple apps; `go_router` is the recommended package for deep links, web URLs, and nested navigation.

DART

```dart
// Imperative: push a new screen and await a result
final result = await Navigator.push(
  context,
  MaterialPageRoute(builder: (_) => const DetailPage(id: 42)),
);

// Return a value from the pushed screen
Navigator.pop(context, 'selected-value');

// Declarative with go_router (add: flutter pub add go_router)
final router = GoRouter(routes: [
  GoRoute(path: '/', builder: (ctx, st) => const HomePage()),
  GoRoute(
    path: '/user/:id',
    builder: (ctx, st) => UserPage(id: st.pathParameters['id']!),
  ),
]);
// MaterialApp.router(routerConfig: router);
context.go('/user/42');   // navigate by URL
```

✦ **When to use go_router**

Choose `go_router` when you need browser-friendly URLs, deep linking from notifications, or route guards (redirects for auth). Stick with raw `Navigator` for a quick two-screen prototype.

Async, Futures & Streams

Dart is single-threaded with an event loop. `Future` represents one eventual value; `Stream` represents many values over time. `async/await` makes both read like synchronous code.

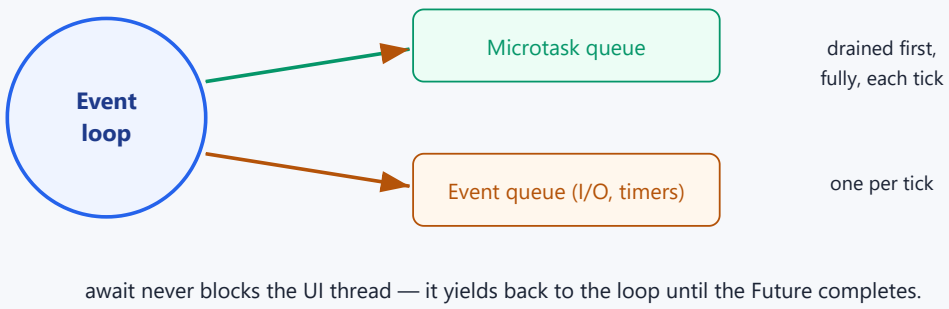


Fig 4. Dart's event loop processes all microtasks before the next event, keeping the UI responsive.

DART

```dart
Future<int> fetchCount() async {
  await Future.delayed(const Duration(seconds: 1)); // simulate I/O
  return 42;
}

Future<void> load() async {
  try {
    final count = await fetchCount();       // suspends, UI stays responsive
    print('Got $count');
  } on TimeoutException {
    print('timed out');
  } catch (e) {
    print('failed: $e');
  } finally {
    print('done');
  }
}

// Run many in parallel and wait for all
final results = await Future.wait([fetchCount(), fetchCount()]);

// Streams: react to many values
Stream<int> ticks() async* {
  for (var i = 1; i ≤ 3; i++) {
    await Future.delayed(const Duration(seconds: 1));
    yield i;                                // emit a value
  }
}
await for (final t in ticks()) { print('tick $t'); }
```

FutureBuilder: async data in the UI

DART

```dart
FutureBuilder<int>(
  future: fetchCount(),
  builder: (context, snapshot) {
    if (snapshot.connectionState ≠ ConnectionState.done) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) return Text('Error: ${snapshot.error}');
    return Text('Count: ${snapshot.data}');
  },
)
```

⚠ **FutureBuilder trap**

Do not create the future inline in `build()` (`future: fetchCount()` as above is for illustration). Every rebuild would restart the request. Store the future in a field assigned in `initState` .

Calling REST APIs

The `http` package covers most needs; `dio` adds interceptors, retries, and cancellation for larger apps.

DART

```dart
import 'dart:convert';
import 'package:http/http.dart' as http;

class Post {
  final int id;
  final String title;
  const Post({required this.id, required this.title});

  // Parse JSON defensively — never trust the shape blindly.
  factory Post.fromJson(Map<String, dynamic> json) => Post(
        id: json['id'] as int,
        title: json['title'] as String,
      );
}

Future<List<Post>> fetchPosts() async {
  final uri = Uri.parse('https://jsonplaceholder.typicode.com/posts');
  final res = await http.get(uri, headers: {'Accept': 'application/json'})
      .timeout(const Duration(seconds: 10));

  if (res.statusCode != 200) {
    throw Exception('HTTP ${res.statusCode}: ${res.reasonPhrase}');
  }
  final List<dynamic> data = jsonDecode(res.body) as List<dynamic>;
  return data
      .map((e) => Post.fromJson(e as Map<String, dynamic>))
      .toList();
}

// POST with a JSON body
Future<void> createPost(String title) async {
  final res = await http.post(
    Uri.parse('https://jsonplaceholder.typicode.com/posts'),
    headers: {'Content-Type': 'application/json'},
    body: jsonEncode({'title': title, 'userId': 1}),
  );
  if (res.statusCode != 201) throw Exception('create failed');
}
```

✦ **Decode off the UI thread**

For large JSON payloads, parse inside `compute()` (which runs the function in a background isolate) so decoding doesn't drop frames.

State Management Overview

Beyond a single widget's `setState`, apps need to share and react to state across the tree. There is no single "right" answer — pick by app size.

Approach	Best for	Notes
`setState`	Local, ephemeral UI state	Built in; scope limited to one widget.
`InheritedWidget` / `Provider`	Sharing values down the tree	Provider wraps InheritedWidget ergonomically.
Riverpod	Most new apps	Compile-safe, testable, no BuildContext needed; successor to Provider.
BLoC / Cubit	Large teams, event-driven	Streams of events to states; very structured.
GetX / MobX / signals	Reactive preferences	Observable-based; concise but opinionated.

DART

```dart
// Riverpod sketch (flutter pub add flutter_riverpod)
final counterProvider = StateProvider<int>((ref) => 0);

class CounterText extends ConsumerWidget {
  const CounterText({super.key});
  @override
  Widget build(BuildContext context, WidgetRef ref) {
    final count = ref.watch(counterProvider);         // rebuilds on change
    return GestureDetector(
      onTap: () => ref.read(counterProvider.notifier).state++,
      child: Text('Tapped $count times'),
    );
  }
}
// Wrap the app once in ProviderScope(child: MyApp()).
```

Recommendation (2026)

For a fresh codebase, start with `setState` for local widgets and **Riverpod** for shared/app-wide state. Reach for BLoC only when the team already standardizes on it.

Worked Mini-Example: A Posts Screen

Putting it together — fetch from a REST API, show a spinner, handle errors, and render a lazy list.

DART

```dart
class PostsPage extends StatefulWidget {
  const PostsPage({super.key});
  @override
  State<PostsPage> createState() => _PostsPageState();
}

class _PostsPageState extends State<PostsPage> {
  late Future<List<Post>> _future;

  @override
  void initState() {
    super.initState();
    _future = fetchPosts();        // created once, not on every build
  }

  Future<void> _refresh() async {
    setState(() => _future = fetchPosts());
    await _future;
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Posts')),
      body: RefreshIndicator(
        onRefresh: _refresh,
        child: FutureBuilder<List<Post>>(
          future: _future,
          builder: (context, snap) {
            if (snap.connectionState == ConnectionState.waiting) {
              return const Center(child: CircularProgressIndicator());
            }
            if (snap.hasError) {
              return Center(child: Text('Failed: ${snap.error}'));
            }
            final posts = snap.data!;
            return ListView.builder(
              itemCount: posts.length,
              itemBuilder: (_, i) => ListTile(
                leading: CircleAvatar(child: Text('${posts[i].id}')),
                title: Text(posts[i].title),
              ),
            );
          },
        ),
      ),
    );
```

```
  }
}
```

Cheat Sheet & Comparison

Task	Snippet
New app	`flutter create app`
Add package	`flutter pub add http`
Run & hot reload	`flutter run` then `r`
Spacer	`SizedBox(height: 16)`
Fill remaining space	`Expanded(child: ...)`
Navigate	`Navigator.push(context, MaterialPageRoute(...))`
Theme color	`Theme.of(context).colorScheme.primary`
Rebuild state	`setState(() => x = y)`
Run tests	`flutter test`

Flutter vs React Native

Dimension	Flutter	React Native
Language	Dart	JavaScript / TypeScript
Rendering	Own engine (Impeller) draws every pixel	Bridges to native OS widgets
UI consistency	Identical across platforms	Follows each platform's native look
Performance	AOT-compiled to native code	JS runtime + bridge (New Architecture improves this)
Hot reload	Yes, stateful	Fast Refresh

Best Practices & Common Pitfalls

✓ **Do**

Prefer `const` constructors everywhere possible — const widgets are cached and skipped during rebuilds. Keep `build()` methods pure and cheap. Extract subtrees into named widgets rather than helper methods returning widgets, so Flutter can rebuild them independently.

✦ **Do**

Use `ListView.builder` (not a `Column` in a `SingleChildScrollView`) for long/infinite lists so items are built lazily. Dispose every controller. Use keys (`ValueKey`) when reordering or inserting into lists.

⚠ **Avoid**

Doing heavy work (JSON parsing, crypto, image decoding) on the UI isolate — offload to `compute()`. Avoid giant widgets; split them up. Avoid rebuilding the whole screen for a tiny change.

✕ **Never**

Call `setState` after `dispose` (guard with `if (mounted)` after an await). Never create expensive objects (controllers, futures) inside `build()`. Never block the main isolate with synchronous I/O.

Interview Questions & Answers

What is the difference between StatelessWidget and StatefulWidget?

A `StatelessWidget` is fully described by its constructor arguments and never changes once built — it has no mutable state. A `StatefulWidget` pairs an immutable widget object with a separate, long-lived `State` object that can hold mutable data and trigger rebuilds via `setState`. Choose stateful only when something must change over the widget's lifetime.

Why are there three trees (Widget, Element, Render)?

Widgets are cheap, immutable configuration recreated on every build. Elements are the persistent instantiation that holds state and position in the tree. Render objects do the expensive layout and

painting. Separating them lets Flutter rebuild widgets freely while reusing the costly render objects, diffing to update only what changed.

Explain Flutter's layout algorithm.

"Constraints go down, sizes go up, parent sets position." A parent passes box constraints (min/max width & height) to each child. Each child picks a size within those constraints and reports it up. The parent then positions the child. This single pass makes layout fast but also explains overflow errors — a child wanted more than the constraint allowed.

What does the `key` parameter do?

Keys preserve widget state when the position in the tree changes — for example reordering list items or swapping between two widgets of the same type. Without a key, Flutter matches elements by type and position and may attach the wrong state. Use `ValueKey` / `ObjectKey` for lists and `GlobalKey` to access state or context across the tree (sparingly).

What is the difference between a Future and a Stream?

A `Future` yields a single asynchronous value (or error) exactly once. A `Stream` yields a sequence of values over time, like websocket messages or sensor events. You `await` a Future and `await for` (or `.listen`) a Stream. In the UI, use `FutureBuilder` for one-shot data and `StreamBuilder` for continuous updates.

How does async work given Dart is single-threaded?

Dart uses an event loop with a microtask queue and an event queue. `await` does not block the thread; it registers a continuation and yields back to the loop, which keeps rendering the UI. When the awaited Future completes, its continuation is scheduled. For true parallelism (CPU-bound work) you spawn an `Isolate` — isolates have separate memory and communicate via message passing.

When would you lift state up or use a state-management library?

Lift state to the nearest common ancestor when two sibling widgets need the same data. When passing it through many layers becomes unwieldy ("prop drilling") or many widgets must react to shared state, reach for Provider/Riverpod/BLoC so state lives outside the widget tree and widgets subscribe to just the slices they need.

What is `BuildContext`?

It is a handle to the location of a widget in the element tree. You use it to look up inherited data (`Theme.of(context)`, `MediaQuery.of(context)`), to navigate (`Navigator.of(context)`), and to find ancestors. A context is only valid for its widget's position — using a stale or wrong-level context is a common bug.

How do you optimize a janky Flutter list?

> Use `ListView.builder` for lazy construction, mark static subtrees `const`, add `cacheExtent` tuning if needed, avoid rebuilding the whole list on each change (use `Selector`/granular providers), keep `itemBuilder` cheap, and move heavy computation to `compute()`. Use the DevTools performance overlay to find the slow frames.

> **What is the difference between hot reload and hot restart?**
>
> Hot reload injects changed Dart source into the running Dart VM and rebuilds the widget tree, *preserving* app state — ideal for UI tweaks. Hot restart recompiles and restarts the app, discarding state — needed for changes to `main()`, global state, enums, or static initializers. Release (AOT) builds have neither.

> **How do you handle errors from a REST call cleanly?**
>
> Wrap the call in try/catch, apply a `.timeout()`, check `statusCode` explicitly, parse JSON through typed `fromJson` factories, and surface failure states in the UI (via a sealed result type or `snapshot.hasError`). Centralize this in a repository layer so widgets only see domain objects or well-typed errors.

> **What is an Isolate and when do you need one?**
>
> An isolate is Dart's unit of concurrency — an independent worker with its own memory and event loop. Because isolates share nothing, there are no data races; they communicate via ports/messages. Use one (often via the `compute()` helper) for CPU-heavy work like parsing large JSON, image processing, or encryption so the UI isolate keeps a smooth 60/120fps.

Further Learning

- **Official docs & codelabs** — flutter.dev and the "Write your first Flutter app" codelab.
- **Dart language tour** — dart.dev; master records, patterns, and sealed classes (Dart 3+).
- **State management** — go deep on Riverpod, then compare with BLoC/Cubit.
- **Navigation** — the `go_router` package for declarative, deep-linkable routing.
- **Testing** — unit, `flutter_test` widget tests, and `integration_test` for end-to-end.
- **Performance** — Flutter DevTools (timeline, memory, widget inspector) and the Impeller renderer.
- **Architecture** — layered app design (presentation / domain / data) and the repository pattern.
- **Platform integration** — platform channels and FFI for calling native code and C libraries.